Week 2, Day 2: AOP, Proxies, Auto-Configuration and Stacks

Prepared for Upendra Kumar Sharma | 3–4 hours | Main topic: Spring AOP, proxies, Spring Boot auto-configuration, starters | DSA: Stacks

Today's Schedule

- **0:00–1:00** Study AOP concepts, proxies and the self-invocation pitfall (sections 1 to 3). Write the timing aspect from section 2 in a Spring Boot project.
- **1:00–1:50** Study auto-configuration, conditions and starters (sections 4 to 6).
- **1:50–2:50** DSA: solve the 3 stack problems (section 7). Spend at most 25 minutes per problem before reading the hint.
- **2:50–3:20** Answer the interview questions aloud (section 8) and add 10 flashcards.

1. AOP Concepts

What problem it solves: cross-cutting concerns such as logging, security checks, transactions, caching, retries and metrics. Without AOP the same code is repeated across many classes and clutters business logic.

Vocabulary

- **Aspect:** a class that packages a cross-cutting concern.
- **Join point:** a point in program execution where an aspect can apply. In Spring AOP this is always a **method execution**.
- **Pointcut:** an expression that selects which join points to match.
- **Advice:** the code to run at a matched join point.
- **Weaving:** linking aspects with the target. Spring AOP does it at runtime by creating proxies.

Advice types

- **@Before:** runs before the method.
- **@AfterReturning:** runs after a normal return, and can see the return value.
- **@AfterThrowing:** runs when the method throws.
- **@After:** runs always, like finally.
- **@Around:** wraps the method and controls whether and when it runs, by calling proceed(). The most powerful type, and the one used for timing, retries and transactions.

Spring AOP vs AspectJ

- **Spring AOP:** proxy-based, applies only to Spring beans, only to method calls that go through the proxy, and is simple to set up.
- **AspectJ:** compile-time or load-time weaving, can advise any object, field access, constructors and self-calls. More powerful, more setup. Use it when Spring AOP's limits get in the way.

2. Writing an Aspect

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface LogExecutionTime {}

@Aspect
@Component
public class TimingAspect {
    private static final Logger log =
LoggerFactory.getLogger(TimingAspect.class);

    @Around("@annotation(com.example.LogExecutionTime)")
    public Object time(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.nanoTime();
        try {
            return pjp.proceed();            // must call proceed(), and
return its result
        } finally {
            long ms = (System.nanoTime() - start) / 1_000_000;
            log.info("{} took {} ms", pjp.getSignature().toShortString(),
ms);
        }
    }
}
```

- Add the **spring-boot-starter-aop** dependency. Spring Boot then enables AspectJ-style proxying automatically, and you do not need @EnableAspectJAutoProxy.
- **Pointcut examples:** execution(* com.example.service...(..)) matches every method in the service packages. @annotation(...) matches methods carrying your annotation. within(...) and bean(...) select by type or bean name.
- **Always rethrow or return correctly in @Around.** If you forget to return the result of proceed(), the caller gets null. If you swallow an exception, you hide failures.
- **Order:** when several aspects apply, control precedence with @Order. A lower value runs first on the way in.

- **Where Spring itself uses AOP:** @Transactional, @Async, @Cacheable, @PreAuthorize and @Retryable are all applied through proxies.

3. Proxies and the Self-Invocation Pitfall

How it works: after a bean is created, a BeanPostProcessor replaces it in the container with a **proxy** that runs the advice and then delegates to the real object. Callers hold the proxy, not the target.

Two kinds of proxy

- **JDK dynamic proxy:** implements the same interfaces as the target. The target must implement at least one interface, and you can only inject it by interface type.
- **CGLIB proxy:** generates a **subclass** of the target at runtime. Works without interfaces. The class and the advised methods **cannot be final**, and private methods cannot be advised.
- **Spring Boot default:** CGLIB (spring.aop.proxy-target-class=true). Injecting by concrete class works.

The self-invocation pitfall

The advice runs only when the call passes through the proxy. A call from one method of a bean to another method of the same bean uses the real object, so no advice is applied.

```
@Service
public class ReportService {
    @Transactional
    public void save(Report r) { /* ... */ }

    public void process(Report r) {
        save(r);          // self-invocation: @Transactional is NOT applied
    }
}
```

Fixes

- Move the advised method into another bean and inject that bean (the cleanest option).
- Inject the bean's own proxy (self-injection with @Lazy), which is awkward but works.
- Use AspectJ weaving mode, which does not rely on proxies.

The same pitfall affects @Async, @Cacheable and @Retryable, and it is one of the most common causes of "my annotation does nothing" bugs. Also remember that advice only applies to **public methods** called from outside the bean, when using proxies.

4. Spring Boot Auto-Configuration

The idea: Boot looks at your classpath, your properties and the beans you have already defined, and configures sensible defaults for what it finds. You override only what you need.

How it works, step by step

1. **@SpringBootApplication** includes **@EnableAutoConfiguration**.
2. That annotation imports an **AutoConfigurationImportSelector**, which reads a list of candidate auto-configuration classes from META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports inside the Boot jars. (Older versions used spring.factories.)
3. Each candidate is a @AutoConfiguration class guarded by **conditions**. Only those whose conditions pass are applied.
4. Because user-defined beans are registered first and most auto-configured beans use **@ConditionalOnMissingBean**, your own beans win. This is the "back-off" behavior.

Example: with spring-boot-starter-data-jpa on the classpath and datasource properties set, DataSourceAutoConfiguration creates a HikariCP DataSource, and HibernateJpaAutoConfiguration builds the EntityManagerFactory. If you define your own DataSource bean, the default one is skipped.

5. Conditional Annotations

- **@ConditionalOnClass / @ConditionalOnMissingClass:** apply only if a class is (or is not) on the classpath.
- **@ConditionalOnBean / @ConditionalOnMissingBean:** apply depending on whether a bean already exists.
- **@ConditionalOnProperty:** apply depending on a property value. The standard way to implement feature flags.
- **@ConditionalOnWebApplication, @ConditionalOnResource, @ConditionalOnExpression** cover other cases.
- **@Profile("dev"):** a simpler condition based on active Spring profiles.

```
@Configuration
public class NotificationConfig {

    @Bean
    @ConditionalOnProperty(name = "notify.sms.enabled", havingValue = "true")
    public MessageSender smsSender() { return new SmsSender(); }

    @Bean
    @ConditionalOnMissingBean(MessageSender.class)
    public MessageSender emailSender() { return new SmtpEmailSender(); }
}
```

Debugging auto-configuration

- Start with **--debug** (or debug=true) to print the **conditions evaluation report**, which lists what matched and what did not, and why.
- With Actuator, the /actuator/conditions endpoint shows the same information.
- Exclude something with @SpringBootApplication(exclude = DataSourceAutoConfiguration.class) or the property spring.autoconfigure.exclude.

6. Starters, Dependency Management and Custom Starters

Starters are curated dependency bundles. One line in your build pulls in a consistent set of libraries.

- **spring-boot-starter-web:** Spring MVC, Jackson, and an embedded Tomcat.
- **spring-boot-starter-data-jpa:** Spring Data JPA, Hibernate and HikariCP.
- **spring-boot-starter-security, -validation, -actuator, -aop, -test** are the other ones you will use constantly.
- **Version management:** the Spring Boot BOM (through spring-boot-starter-parent or the dependency-management plugin) pins compatible versions, so you rarely specify versions for Boot-managed libraries.
- **Switching the embedded server:** exclude spring-boot-starter-tomcat and add spring-boot-starter-jetty or -undertow.

Externalized configuration, briefly

- Properties come from application.properties or application.yml, profile-specific files (application-prod.yml), environment variables and command-line arguments. Command-line arguments and environment variables override file values.
- **@ConfigurationProperties** binds a group of properties to a typed class, with validation. It is better than scattering @Value for related settings.

Writing your own starter (interview favorite)

1. Create an autoconfigure module with a @AutoConfiguration class that defines beans using @ConditionalOnMissingBean, plus a @ConfigurationProperties class for settings.
2. List the class in META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports.
3. Create a thin starter module that depends on the autoconfigure module and the libraries it needs.
4. By convention, third-party starters are named acme-spring-boot-starter, since spring-boot-starter-* is reserved for official ones.

```
@AutoConfiguration
@EnableConfigurationProperties(AuditProperties.class)
@ConditionalOnProperty(prefix = "audit", name = "enabled", havingValue =
"true", matchIfMissing = true)
public class AuditAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean
    public AuditService auditService(AuditProperties props) {
        return new AuditService(props.getTopic());
    }
}
```

7. DSA: Stacks (3 Problems)

Use **ArrayDeque** as a stack (push, pop, peek), not the legacy Stack class.

Problem 1: Valid Parentheses

- **Idea:** push the expected closing bracket for every opening bracket. For a closing bracket, pop and compare. The string is valid if every comparison matches and the stack is empty at the end.
- **Complexity:** O(n) time, O(n) space.
- **Edge cases:** empty string (valid), a closing bracket first (stack is empty), leftover opening brackets, odd length.

```
public boolean isValid(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(') stack.push(')');
        else if (c == '[') stack.push(']');
        else if (c == '{') stack.push('}');
        else if (stack.isEmpty() || stack.pop() != c) return false;
    }
    return stack.isEmpty();
}
```

Problem 2: Min Stack

- **Idea:** store each value together with the minimum seen so far. Every operation, including getMin, is then O(1). An alternative is a second stack that tracks minimums.
- **Complexity:** O(1) per operation, O(n) space.
- **Edge cases:** duplicate minimums, popping the current minimum, negative values.

```
class MinStack {
    private final Deque<int[]> stack = new ArrayDeque<>();   // {value,
minSoFar}

    public void push(int val) {
        int min = stack.isEmpty() ? val : Math.min(val, stack.peek()[1]);
        stack.push(new int[]{val, min});
    }
    public void pop()    { stack.pop(); }
    public int top()     { return stack.peek()[0]; }
    public int getMin()  { return stack.peek()[1]; }
}
```

Problem 3: Daily Temperatures

- **Idea:** a **monotonic stack** of indices with decreasing temperatures. When today's temperature is higher than the temperature at the top index, pop it and set its answer to the difference in days.
- **Complexity:** O(n) time (each index is pushed and popped once), O(n) space.
- **Edge cases:** strictly decreasing temperatures (all zeros), equal temperatures (not warmer), a single day.
- **Pattern to remember:** "next greater element" problems use a monotonic stack.

```
public int[] dailyTemperatures(int[] temps) {
    int n = temps.length;
    int[] answer = new int[n];
    Deque<Integer> stack = new ArrayDeque<>();           // indices
    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && temps[i] > temps[stack.peek()]) {
            int j = stack.pop();
            answer[j] = i - j;
        }
        stack.push(i);
    }
    return answer;
}
```

8. Interview Questions to Answer Aloud

1. What is AOP, and what problems does it solve? Name the key terms.
2. What are the advice types, and why is @Around the most powerful?
3. How does Spring AOP differ from AspectJ?
4. JDK dynamic proxy vs CGLIB proxy: how do they differ, and which does Spring Boot use by default?
5. Why does calling a @Transactional method from another method of the same class not start a transaction?
6. What limits does a CGLIB proxy have (final classes and methods, private methods)?
7. Where does Spring create proxies in the bean lifecycle?
8. How does Spring Boot auto-configuration work internally?
9. What does @ConditionalOnMissingBean do, and why do your own beans override defaults?
10. How do you find out why an auto-configuration did or did not apply?
11. What is a starter? How would you write your own?
12. How do you exclude an auto-configuration, and how do you switch from Tomcat to Jetty?
13. @ConfigurationProperties vs @Value: when do you use each?

9. End-of-Day Checklist

- [ ] I can explain AOP terms and write an @Around aspect with a custom annotation.
- [ ] I can explain JDK vs CGLIB proxies and the self-invocation pitfall with its fixes.

- [ ] I can describe how auto-configuration is discovered and how conditions make it back off.
- [ ] I can list five conditional annotations and say when to use each.
- [ ] I can outline how to build a custom starter.
- [ ] I solved all 3 stack problems and can explain the monotonic stack pattern.
- [ ] I tried the aspect and the --debug conditions report in a real project.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Week 2, Day 3): @Transactional propagation and isolation, rollback rules and pitfalls, plus interval problems.